

Scalability & Future Plan

Date	04-July-2026
Team ID	XXXXXX
Project Name	Smart Lender
Maximum Marks	1 Mark

Scalability & Future Plan:

This document outlines how the current project solution can be scaled to handle larger user bases, increased data loads, or extended features in the future. It also captures the team's roadmap for enhancing and evolving the project beyond its current state.

Current System Limitations:

S.No	Limitation	Impact	Priority to Address (High / Medium / Low)
1	Standalone Local Hosting Loop	The application runs on a local machine (127.0.0.1:5000), restricting access for multiple remote credit underwriters.	Medium
2	Dynamic In-Memory States	User parameters are processed instantly but vanish upon page reload because there is no historical database tracking.	Low
3	Single-Model Estimation Matrix	System processing relies entirely on a single optimized Random Forest ensemble logic.	Low

Scalability Plan:

S.No	Scalability Aspect	Current State	Proposed Upgrade / Solution
1	User Load	Dev-server setup restricted to local standalone testing.	Bind the Flask core logic to a multi-threaded execution server or standard gateway like Gunicorn to easily manage concurrent requests.
2	Data Storage	Local workspace repository relying entirely on static file serialization (rdf.pkl).	Connect the submission route handler directly to a standard relational schema (such as SQLite or PostgreSQL) to safely retain transaction logs over time.
3	Performance	In-memory prediction execution mapping returns under 0.2 seconds locally.	Implement a lightweight caching structure to handle identical duplicate parameter checks instantly, preserving compute cycles.
4	Security	Payload validation is fully locked down using browser form rules and strict data-type casting enforcements.	Deploy standard Transport Layer Security encryption (HTTPS/SSL protocols) to securely protect input strings across a hosted environment.

Future Roadmap:

Phase	Planned Feature / Enhancement	Target Timeline	Expected Impact
Phase 2	Public Cloud Infrastructure Integration: Migrate the functional local Flask system directly to a public cloud cloud framework (such as Render or Vercel) using unified Git workflows.	Q4 2026	Transitions the platform from a local tool to a globally accessible microservice, allowing institutional stakeholders to access the automated credit underwriting portal seamlessly from any remote browser.